

DBASC IT SUPPORT

IT Asset & Help Desk Management System

Complete Project Specification

A production-ready college IT management platform for 300+ IT assets, staff support tickets, asset tracking, reporting, and role-based access.

1. Project Overview

The system will provide one central portal for managing college IT assets and internal IT support. The System Administrator has full control, staff can create and track their own tickets, and the Principal receives read-only dashboards and reports.

2. Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, Vanilla JavaScript ES6+, Bootstrap 5, Bootstrap Icons
Charts/UI	Chart.js, DataTables, SweetAlert2
Backend	Node.js + Express.js / Vercel-compatible API functions
Database	Supabase PostgreSQL
Authentication	Supabase Auth
File Storage	Supabase Storage
Email	Resend or equivalent transactional email provider
QR Codes	qrcode npm package
Hosting	Vercel
Source Control	GitHub

3. User Roles & Permissions

Role	Permissions
Admin / System Admin	Full access to assets, tickets, users, departments, locations, reports, QR codes, imports/exports, settings and au
Staff / Faculty	Create tickets, view own tickets, reply to own tickets, upload attachments and track status. No asset administrati
Principal	Read-only dashboards, asset statistics, ticket statistics and reports. No editing/deleting/assignment.

4. Core Modules

- **Dashboard:** asset and ticket KPIs, charts, trends and alerts.
- **Asset Management:** complete lifecycle records for PCs, laptops, servers, switches, routers, UPS, printers, projectors, smart boards, APs, CCTV and other equipment.
- **Help Desk:** ticket creation, assignment, status workflow, comments, attachments and resolution history.
- **User Management:** staff, principal and admin accounts with role-based permissions.
- **Reports:** asset, warranty, department, location, ticket and resolution reports.
- **QR Management:** QR code for every asset with printable labels.
- **Audit Logs:** record important administrative and ticket/asset changes.
- **Bulk Import:** CSV import with validation and duplicate detection.

5. Asset Management Specification

Asset categories: Desktop PC, Laptop, Server, Switch, Router, Firewall, UPS, Printer, Projector, Smart Board, Access Point, CCTV, NVR, Monitor, Keyboard, Mouse and Other.

Asset fields: Asset ID, name, category, brand, model, serial number, processor, RAM, storage, operating system, purchase date/cost, warranty dates, vendor, location, department, assigned user, status, condition, IP, MAC, hostname, notes and image.

Statuses: Active, In Repair, Spare, Retired, Scrapped, Lost. Conditions: Excellent, Good, Fair, Damaged.

Asset IDs: DB-PC-0001, DB-LAP-0001, DB-SRV-0001, DB-SW-0001, DB-UPS-0001, etc.

6. Help Desk Specification

Ticket fields: Ticket number, title, description, category, priority, status, requester, department, location, asset, assigned technician, resolution, timestamps and attachments.

Priorities: Low, Medium, High, Critical.

Status workflow: Open → Assigned → In Progress → Waiting for User → Resolved → Closed.

Ticket IDs: DB-HD-000001, DB-HD-000002, etc.

7. Database Design

- users — profile, role, department and account status.
- departments — department master data.
- locations — building, floor and room/location data.
- asset_categories — asset type master data.
- assets — complete IT asset records.
- asset_assignments — assignment history.
- asset_service_history — repair and maintenance history.
- tickets — help desk tickets.
- ticket_categories — issue categories.
- ticket_comments — ticket conversation.
- ticket_attachments — uploaded files.
- notifications — user notifications.
- audit_logs — security and administrative audit trail.

8. Recommended College Master Data

Departments: BCA, B.Com, Viscom, B.Sc CS AI, Administration, Library, Accounts and other departments as required.

Locations: Lab 1, Lab 2, Lab 3, Server Room, Office, Library, Principal Room, Classrooms and other rooms.

9. Frontend Pages

Role	Pages
Admin	Dashboard, Assets, Asset Details, Add/Edit Asset, Tickets, Ticket Details, Users, Departments, Locations, Reports, A
Staff	Dashboard, Create Ticket, My Tickets, Ticket Details, Profile
Principal	Dashboard, Asset Overview, Ticket Overview, Reports, Profile
Common	Login, Forgot Password, Error 403, Error 404, Error 500

10. Dashboard Requirements

- Admin: Total Assets, Active Assets, Under Repair, Retired, Open Tickets, In Progress, Resolved and Closed.
- Charts: asset distribution, ticket status, tickets by department and 12-month ticket trend.
- Staff: My Open, In Progress, Resolved and Closed tickets plus Create Ticket shortcut.
- Principal: executive read-only view with asset and ticket KPIs, department trends, monthly trends and reports.

11. Search, Filtering & Pagination

Assets must support search by asset ID, serial number, brand, model, hostname, IP, MAC, location, department and assigned user. Tickets must support search by ticket number, title, requester, asset and department, with filters for status, priority, category and date range. Use server-side pagination for 300+ assets and thousands of tickets.

12. QR Codes & Asset Labels

Every asset receives a QR code containing the asset ID or secure asset URL. Admin can view, download and print QR labels. Sensitive information must not be exposed to unauthorized users.

13. Notifications & Email

- Staff: ticket created, assigned, status changed, resolved and admin reply.
- Admin: new ticket and high/critical ticket alerts.
- Optional Principal: monthly IT report.
- Use environment variables for email API credentials.

14. Reports

- All assets, department-wise, location-wise, category-wise and status-wise asset reports.
- Warranty expiry and repair/service reports.
- Monthly tickets, department-wise tickets, category-wise tickets and priority-wise tickets.
- Average resolution time and technician performance.
- Export to CSV/Excel-compatible CSV and PDF.

15. Security

- Supabase Auth for authentication.
- Supabase Row Level Security for database authorization.
- Server-side role verification for every protected operation.
- Never expose SUPABASE_SERVICE_ROLE_KEY to frontend.

- Input validation, secure headers, rate limiting and safe file validation.
- Do not allow executable file uploads.
- Audit important actions including login, asset changes, ticket changes and user changes.

16. Supabase Storage

Recommended private buckets: **ticket-attachments**, **asset-images**, and **asset-documents**. Configure policies so users only access files permitted by their role and ticket/asset relationship.

17. Vercel Deployment Architecture

GitHub → Vercel → Frontend/API → Supabase PostgreSQL/Auth/Storage

The Node/Express backend must be implemented in a Vercel-compatible serverless/API structure rather than relying on a permanent `app.listen()` process. Environment variables must be configured in Vercel.

18. Suggested Project Structure

```
/project /public /css /js /images index.html login.html /api /assets /tickets /users /reports /server
auth.js supabase.js middleware.js validation.js email.js /database schema.sql seed.sql rls.sql /utils
qr.js reports.js .env.example .gitignore package.json vercel.json README.md
```

19. Bulk Asset Import

Support CSV import with preview, validation, duplicate detection, error reporting and safe transaction handling.

Suggested columns: `asset_code`, `asset_name`, `category`, `brand`, `model`, `serial_number`, `processor`, `ram`, `storage`, `operating_system`, `purchase_date`, `warranty_end`, `location`, `department`, `assigned_user`, `ip_address`, `mac_address`, `hostname`, `status` and `condition`.

20. Performance Target

Design for at least 500+ assets, 100+ staff users and thousands of tickets. Use indexes, pagination, server-side filtering, optimized queries and image optimization.

21. Implementation Workflow

- Create GitHub repository.
- Create Supabase project.
- Create PostgreSQL schema and indexes.
- Configure Supabase Auth.
- Create RLS policies for Admin, Staff and Principal.
- Create Storage buckets and policies.
- Build frontend pages and reusable UI components.
- Build Vercel-compatible Node.js API endpoints.
- Connect frontend to APIs/Supabase.
- Implement asset CRUD and QR generation.
- Implement ticket workflow and comments.
- Implement role-based dashboards.
- Implement reports and exports.
- Implement email notifications.
- Add audit logging.
- Test all three roles and security rules.
- Deploy to Vercel.
- Configure custom domain if required.
- Perform production backup/recovery testing.

22. Final Acceptance Criteria

- Admin can fully manage 300+ assets and tickets.
- Staff can create and track only their own tickets.
- Principal has read-only access to required dashboards and reports.
- Asset IDs and ticket IDs are unique and automatically generated.
- QR codes work for assets.
- Attachments are stored securely in Supabase Storage.
- RLS and server-side authorization prevent unauthorized access.
- CSV import/export works with validation.
- Reports work on desktop and mobile.
- Application contains no hard-coded secrets.
- Application deploys successfully on Vercel.
- README contains setup, environment variables, database, RLS and deployment instructions.

23. Example Ticket Workflow

Staff submits: **Internet not working in Lab 2** → system creates DB-HD-000125 → Admin receives notification → Admin assigns/works on ticket → Admin adds resolution → Staff verifies → ticket becomes Resolved/Closed → history

remains available for reporting.

24. Example Asset Workflow

Admin creates DB-PC-0001 → records serial number/specifications → assigns Lab 1 and department → prints QR label → asset is later repaired or reassigned → every important change is recorded in asset history → asset can eventually be retired.

25. Recommended Final Stack

Frontend: HTML + CSS + Vanilla JavaScript + Bootstrap 5

Backend: Node.js + Express.js/Vercel API

Database: Supabase PostgreSQL

Authentication: Supabase Auth

Storage: Supabase Storage

Email: Resend

Hosting: Vercel

Repository: GitHub

Project goal: Deliver a professional, secure and scalable college IT Asset + Help Desk system that can start with 300+ assets and grow to the entire campus.